

HOLY ANGEL UNIVERSITY
School of Computing
6ADET
Final Project Proposal (Revised)

A Case Study Presented to the
Faculty of Mathematics and Science,
School of Arts and Sciences
Holy Angel University
Angeles City



Submitted by:
Allen David C. Panganiban

CS-302

August 2026

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

Bahantabay

Community-Based Flood Monitoring and Route Warning App

1. App Name

Bahantabay — Community-Based Flood Monitoring and Route Warning App.

The app name remains unchanged from the preliminary proposal because it still accurately describes the project's purpose.

2. The Problem, in One Sentence

Commuters in flood-prone areas of the Philippines find out a specific road, underpass, or intersection is impassable only after they've already left, because weather apps report citywide rainfall forecasts rather than the real-time passability of the exact streets they travel.

This is unchanged from the prelim. Re-reading it, it still describes a specific situation (a particular road being impassable) rather than a generic problem statement, so I kept it as written.

3. Who This Is For

Primary users:

Daily commuters in Angeles City and nearby flood-prone cities who travel the same 1–3 routes repeatedly during the rainy season, specifically students commuting to school, office workers, and motorcycle/delivery riders who need to decide before leaving the house whether their usual route is passable.

What they currently do instead:

They check general weather apps for rainfall forecasts, ask in Facebook group chats or community pages whether anyone has passed through recently, or simply leave and discover the flooded road themselves once already stuck in traffic with no practical way to reroute. These methods provide general weather information or informal and inconsistently verified reports, rather than a route-specific indication of whether a particular road is currently passable.

4. Core Features (MVP), Revised

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

Having built M4A4 and M5A5, I now have a better understanding of the time required to implement screens involving real state, forms, lists, navigation, and validation. I re-scored the original feature list against that experience. I also separated route flood-status calculation from the Route Details screen because it is a distinct piece of application logic rather than only a UI task.

All five features remain in the MVP because they represent the minimum workflow that makes Bahantabay function as a community-based flood monitoring and route warning application. However, I deliberately simplified the implementation of route creation and moved optional photo attachments to the stretch-goal list so that the core application remains achievable within the remaining development time.

#	Feature	Still in MVP?	Flutter pieces it needs	Honest estimate
1	Authentication / guest access	Keep	Form, TextFormField, TextEditingController, ElevatedButton, Supabase Auth (signUp, signInWithPassword, signOut), go_router authentication redirects, guest-mode state	7 hours
2	Home: saved routes + nearby flood reports	Keep	Scaffold, AppBar, IconButton / PopupMenuButton for account actions, SegmentedButton for the List/Map view toggle, Card, ListView.builder, Supabase select queries, FlutterMap, MarkerLayer	9 hours
3	Save a route (Add Route)	Keep, simplified	Form, TextFormField, TextEditingController, FlutterMap, map onTap coordinate selection, MarkerLayer, ElevatedButton, Supabase insert	9 hours
4	Report a flood	Keep	Form, TextFormField, DropdownButtonFormField, ChoiceChip/Radio, FlutterMap, geolocator permission handling, ElevatedButton, Supabase insert	9 hours
5	Route flood status (Route Details)	Keep	Custom Dart point-to-line-segment distance calculation using geographic coordinates, PolylineLayer, MarkerLayer, status-badge widget, ListView.builder	7 hours

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

	Total (5 MVP features)			41 hours
--	-------------------------------	--	--	-----------------

The profile/account menu and sign-out action are part of the existing Authentication and Home implementations rather than separate MVP features. The menu is an overlay, not an additional screen, so the five-screen scope remains unchanged.

Scope decisions

The route and map features were simplified rather than removed because they are central to Bahantabay's purpose. Removing saved routes or community flood reporting would remove the application's main use case rather than simply reduce its scope.

For the revised MVP, Add Route no longer depends on a road-routing or Directions API. Instead, the user selects a starting point and destination directly on the map, and those coordinates are saved to the routes table. For the MVP's route-status calculation, the saved route will be represented as a simplified geographic line segment between the user's selected starting point and destination. This does not attempt to reproduce the actual road geometry between the two points; it is an intentional simplification that allows the application to demonstrate route-based flood checking without introducing the additional implementation and API-management cost of a third-party routing service. Real road-following routing can remain a future stretch goal if time permits.

The map is also intentionally limited to the functionality required by the MVP: displaying the selected route representation, allowing users to select locations, and displaying flood-report markers. Turn-by-turn navigation, address search, and other advanced mapping functions are outside the current scope.

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

The five feature estimates total 41 hours. In addition, I expect approximately 4 hours for the initial Supabase project, PostgreSQL database, and Row Level Security setup. Navigation and theme integration will be completed as part of the five feature implementations rather than counted as a separate feature, avoiding double-counting. This gives an estimated core implementation effort of approximately 45 hours, excluding stretch goals.

These are revised estimates based on the pace and complexity I experienced while completing those activities rather than measured historical totals. If further scope reduction becomes necessary, photo attachment will remain a stretch goal because the flood-reporting workflow is fully functional without it.

5. Stretch Goals

The following features are intentionally outside the MVP and will only be implemented if the core application is completed with sufficient time remaining. They are listed in priority order:

1. **Real road-following route geometry** — use a routing service such as OSRM or OpenRouteService to generate a road-following path between the selected starting point and destination instead of the simplified straight-line route used in the MVP. This would improve the accuracy of route-based flood matching but would introduce an additional external service, network dependency, response handling, and possible rate-limit or availability issues.
2. **Push notifications** — notify users when a new nearby flood report affects one of their saved routes. This would require notification services and additional background/event-handling logic beyond the MVP.

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

3. **Flood-severity visualization** — add a simple color-based flood-severity layer or heatmap using recent flood reports to make areas with multiple or more severe reports easier to identify visually.
4. **Trusted reporter indicator** — identify users whose flood reports are frequently corroborated by other reports, providing an additional indication of report reliability.
5. **Advanced map interaction** — add features such as address or place search, draggable route markers, and more convenient location selection beyond the MVP's simple tap-to-place approach.
6. **Flood-report photo attachment** — allow users to attach a photo to a flood report using `image_picker` and Supabase Storage. Photo uploading is intentionally excluded from the MVP because a report can still provide useful flood information through its location, depth category, road-status selection, and timestamp without an image. If implemented later, additional improvements such as image compression and multiple-photo support could also be considered.

These stretch goals reflect features that were either moved out of the original scope during revision or identified as useful enhancements after reconsidering the project's implementation cost. The MVP is deliberately limited to the functionality required to save routes, monitor nearby flood reports, submit community reports, and determine a simplified flood status for a saved route. Completing the MVP takes priority over implementing any stretch goal.

6. How My App Saves Data

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

Shared-data question:

Yes. If two different people install Bahantabay, they must be able to see the same community flood reports. A flood report submitted by one commuter has value specifically because another commuter, using a different device, can view that report before deciding whether to travel. Saved routes are different: they are private to the user who created them and must not be visible in another user's account. Because flood reports need to be shared across users and devices, the core application cannot rely on local-only storage.

Realistic record volume:

This is a planning estimate rather than measured production data. During active development and testing with myself and a small group of classmates, I expect approximately 20–100 flood reports per week, depending on how frequently the reporting workflow is tested, with approximately 1–5 saved routes per user. Over the full development and demonstration period, this could result in a few hundred flood reports in total. This is a relatively small dataset, but it is structured, relational, and shared between users, making a cloud database more appropriate than simple local storage.

My choice: Supabase

I will use Supabase as the backend for Bahantabay, with its PostgreSQL database as the application's primary persistent data store.

Why Supabase and not the alternatives:

- `shared_preferences` — rejected. It is local key-value storage intended for data that belongs to one device. A flood report saved this way would not automatically become available to another user's device, so it cannot satisfy Bahantabay's community-sharing requirement.
- Hive — rejected for the core data. Hive is a local on-device database and is useful for applications that need larger amounts of data without a server. However, it does not by itself provide the multi-user synchronization required for Bahantabay. Using it for shared flood reports would still require building a separate backend and synchronization system.

HOLY ANGEL UNIVERSITY**School of Computing****6ADET**

- Drift / sqflite — rejected for the core data model. Both are suitable for local relational data, but the data would remain on the individual device. To make flood reports shared between users, I would still need to build a separate server or synchronization layer. This would add complexity without providing an advantage over using a backend designed for shared data.
- Firebase — considered but not selected. Firebase Authentication and Firestore or Realtime Database could support Bahantabay's multi-user requirements, so Firebase is a valid alternative rather than an unsuitable technology. I selected Supabase because Bahantabay's data has a naturally relational structure: users have saved routes, users submit flood reports, and each report belongs to a specific reporter. Supabase provides a PostgreSQL relational database and SQL-based querying, which fits this structure and allows me to continue working with relational database concepts.

Supabase is therefore not being selected because it is universally better than Firebase. It is being selected because PostgreSQL's relational model is a good match for Bahantabay's data and allows the project's shared data to be organized using tables, relationships, queries, and Row Level Security.

Tradeoff accepted

I accepted the additional setup complexity of using a cloud backend, authentication, database schema design, and Row Level Security because shared community data is a core requirement of Bahantabay. A local database would be simpler to implement, but it would not allow a flood report submitted by one user to become available to another user on a different device. The additional backend complexity therefore directly supports one of the application's main purposes rather than being added only for technical sophistication.

What I save, concretely (planned Supabase schema — not yet implemented):

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

Table	Fields	Purpose
profiles	id, display_name, created_at	Stores basic user profile information. Authentication credentials are handled by Supabase Auth rather than stored in this table.
routes	id, user_id, name, start_latitude, start_longitude, destination_latitude, destination_longitude, created_at	Stores each user's saved routes and the two geographic points used to represent the simplified MVP route.
flood_reports	id, reporter_id, latitude, longitude, flood_depth, road_status, notes, created_at	Stores community-submitted flood information that can be viewed by other users.

The MVP does not store flood-report photos. Photo attachment is a stretch goal. If that feature is implemented later, the image file will be stored using Supabase Storage, while the corresponding file URL can be stored in flood_reports.photo_url.

Supabase Auth will manage authentication credentials such as passwords. The application will therefore not create its own password field in the profiles table.

Have I tried it yet?

No. I have not yet completed the one-hour Supabase spike. I will complete it by August 29, 2026 by creating a Supabase project, creating one test table, inserting one test record, reading the record back through a small throwaway Flutter test screen, and confirming that the application can connect to Supabase and perform basic read/write operations.

I will also verify that the client application does not contain a Supabase service-role key and that the database access rules work as intended. This spike will be completed before implementing the five MVP features so that I can identify connection, authentication, database, or Row Level Security problems while the project is still small and easy to change.

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

7. One Thing I Want to Add That the Course Did Not Teach

Feature: Interactive map visualization with location markers and current-location detection.

M4 and M5 taught layout, theming, state management, forms, lists, and basic Flutter application structure, but they did not cover integrating an interactive map or obtaining the device's geographic location. Bahantabay depends on spatial information because routes and flood reports are represented by geographic coordinates. Therefore, an interactive map is not simply a visual enhancement; it is necessary for the MVP features involving route creation, route visualization, and flood-report locations.

Packages: I plan to use flutter_map for the interactive map, latlong2 for latitude/longitude coordinate handling, and geolocator for obtaining the device's current location.

For mapping, flutter_map will use OpenStreetMap-based map tiles and will run in the web environment used for development and demonstration. It does not require native platform code or a Google Maps API key for the planned MVP map functionality. geolocator also provides web support through the browser's Geolocation API, although browser location access requires user permission and may be less consistent than location access on an Android device.

Core or stretch: The interactive map itself is a core MVP feature because the application needs a practical way to select route endpoints, display the saved route representation, and visualize flood-report locations. It will therefore be implemented early rather than treated as an optional enhancement.

Current-location detection through geolocator is a supporting enhancement within the Report Flood feature, rather than a requirement for the application to function. The reporting screen will provide a manual map-based location-selection fallback so that a user can still submit a report if

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

location permission is denied, location services are unavailable, or browser location access does not work. This also ensures that the complete application remains usable in the browser.

I will develop and test the browser-compatible map and its basic interactions as part of the main MVP implementation. I also plan to demonstrate the actual GPS-based location behavior on an Android device in the final project video, while the browser demonstration will use the manual map-selection path when real device location is unavailable.

8. How My Project Runs When Someone Else Opens It

Device Preview: Yes. I am keeping the `device_preview` wrapper from the M4/M5 starter project because it remains useful for checking the application's layout at different device sizes. There is no current feature that requires removing it.

My app runs in a browser with flutter run -d web-server, start to finish, with every screen reachable: Not yet. I previously confirmed that the Flutter project can run successfully in a web environment, but I have not yet verified the complete revised application because the Supabase integration and five MVP screens are still being implemented. I will verify the complete browser flow using `flutter run -d web-server` as the screens are completed, making sure that Sign In/Guest Entry, Home, Add Route, Route Details, and Report Flood can all be reached without dead ends or crashes.

Anything that needs real hardware degrades to sample/manual data instead of crashing: Planned. The main hardware-dependent feature is GPS location through geolocator. If browser or device location is unavailable or permission is denied, Report Flood will provide a manual map-tap method for selecting the report location. Therefore, the reporting workflow will remain usable without GPS. Photo attachment is also optional, so a report must still submit successfully when no

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

photo is provided. The browser demonstration will use these fallback paths where necessary, while actual GPS behavior can be demonstrated separately on an Android device.

Public repository and secrets: The project will live in a public repository in my own GitHub account, not in the course organization. The application will require the Supabase project URL and Supabase anonymous/public key for its client-side connection. These are intended for use by client applications and are not treated as substitutes for database security; access to the database will instead be controlled through Supabase Row Level Security policies. I will keep the configuration values out of ordinary source files during development by using a git-ignored .env file locally and appropriate repository/deployment configuration when needed.

The Supabase service-role key will never be placed in the Flutter application or committed to the public repository, because it has elevated privileges that can bypass Row Level Security. The application has no legitimate reason to expose or use this key on the client. No real personal information, passwords, or private user data will be included in the public repository, seed data, or screenshots.

9. Data the App Remembers

Thing	Fields	Where it is saved
User profile	id, display_name, created_at	Supabase PostgreSQL — profiles table
Authentication account	Email and authentication credentials/session information managed by Supabase Auth	Supabase Auth — built-in authentication system
Saved route	id, user_id, name, start_latitude, start_longitude, destination_latitude, destination_longitude, created_at	Supabase PostgreSQL — routes table

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

Flood report	id, reporter_id, latitude, longitude, flood_depth, road_status, notes, created_at	Supabase PostgreSQL — flood_reports table
--------------	---	--

10. Screens

The screen list is unchanged from the prelim — five screens, no more, no fewer. I reconsidered whether to split or merge any of them after building M4A4/M5A5, and concluded the original split was already correct: each screen maps to one clear user task (authenticate, browse, create a route, check a route's status, submit a report), which matches how M5A5 taught me to separate a form-heavy task from a browse/list task rather than crowding both into one screen.

Screen	Purpose	Displays	Reads	Writes	Key widgets/packages
Sign In / Guest Entry	Single entry point for identity	Email/password form and guest entry option	Nothing from the application database	Nothing directly — calls Supabase Auth, which creates the session	Form, TextFormField, TextEditingController, ElevatedButton, Supabase Auth
Home	Daily dashboard	Bahantabay app bar with profile/account access, centered List/Map toggle, saved routes and nearby flood reports in List View, or route/report visualization in Map View	routes (mine), flood_reports (nearby)	Nothing to the application database; account menu may call Supabase Auth signOut	Scaffold, AppBar, IconButton / PopupMenuButton, SegmentedButton, Card, ListView.builder, FlutterMap, MarkerLayer

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

Add Route	Create and save a route	Route name field, map with two tappable points	Nothing from the database	One new record in routes	Form, TextFormField, TextEditingController, FlutterMap, map onTap, MarkerLayer, ElevatedButton
Route Details	Check one saved route's flood status	Simplified route line, nearby flood-report markers, and Safe/Warning/Not Passable status	One saved route and relevant flood_reports within the route-distance threshold	Nothing	FlutterMap, PolylineLayer, MarkerLayer, status-badge widget, ListView.builder
Report Flood	Submit a community flood report	Location selection, flood-depth picker, road-status selection, and optional notes	Device location through geolocator when permission is available	One new record in flood_reports	Form, TextFormField, DropdownButtonFormField, ChoiceChip/Radio, FlutterMap, geolocator, ElevatedButton

Home account access: The profile icon in the Home app bar opens a small Material account menu rather than a separate Profile screen. A signed-in user can log out or switch accounts; either action signs out of the current Supabase session and returns to Sign In / Guest Entry. A guest user can choose to sign in or create an account through the same authentication entry screen. Because this interaction uses an overlay, Bahantabay still contains exactly five MVP screens.

11. Risks, Revised

Risk 1 (revisited): Route/flood matching logic

This was the only risk I identified in the preliminary proposal. Originally, the risk involved determining whether a flood report matched a route represented by a Google Directions polyline containing many coordinate points. After reconsidering the route implementation, the MVP now

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

represents a saved route using only its start and destination coordinates rather than a full road-following polyline. This makes the implementation smaller because the matching logic can use a point-to-line-segment distance calculation instead of comparing the flood report against hundreds of route points.

However, the underlying uncertainty is still a risk. I still do not know what distance threshold will correctly distinguish a flood report that affects the saved route from a report located on a nearby but unrelated street. GPS coordinates also have some amount of positional error, so a threshold that is too small could miss relevant reports while a threshold that is too large could produce false warnings. Therefore, the risk became smaller in implementation complexity but was not eliminated by the revised route representation.

First step: I will create a small test dataset containing approximately 5–10 flood-report coordinates and 2–3 known start/destination route pairs. I will test several candidate thresholds, such as 50 m, 100 m, and 150 m, and compare the resulting matches visually on the map. I will store the selected threshold as a named constant so that it can be adjusted during testing without changing the matching algorithm itself.

Target date: I will complete this test before implementing the Route Details screen, because the screen depends directly on the route/flood matching calculation.

Risk 2 (new): Supabase / backend integration

This is a new risk because the preliminary proposal used Serverpod, while the revised proposal now uses Supabase. The new backend introduces uncertainty around Supabase Authentication, the PostgreSQL database schema, relationships between users, routes, and flood reports, Row Level Security policies, asynchronous loading and error states in Flutter, and ensuring that no privileged credentials are accidentally exposed in the public GitHub repository.

Row Level Security is particularly important because a policy that is incorrectly configured could prevent the application from reading or writing data even when the Flutter code itself appears correct. This could be difficult to diagnose if I wait until the complete application has already been built.

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

First step: I will complete the one-hour Supabase spike described in Section 6 by August 29, 2026. I will create the Supabase project, create one test table, insert a test record, read the record through a small Flutter test screen, and verify that the client can perform the required operation without using a Supabase service-role key. This will allow me to identify authentication, database, or RLS problems before they become dependencies of the five MVP screens.

12. What Changed, and Why

Section	Prelim said	Now says	Why it changed
Backend / persistence	Serverpod + PostgreSQL, self-hosted with Docker and Redis	Supabase (Auth + PostgreSQL + optional Storage)	I reconsidered the backend after reviewing the persistence requirements for the revised proposal. Bahantabay needs shared flood reports that can be accessed by different users, while saved routes should remain associated with their individual users. Supabase provides a hosted PostgreSQL database, authentication, and Row Level Security in one service, which lets me keep the relational data model without having to build and maintain as much backend infrastructure myself.
Authentication	User authentication handled through Serverpod's built-in authentication, with passwords managed by the backend	Supabase Auth manages sign-up, sign-in, sign-out, and authentication sessions; the application's own tables do not store passwords.	This changed as a direct consequence of changing the backend. Supabase Auth now handles the complete authentication lifecycle, while the application's own profiles, routes, and flood_reports tables store only the application data they need.

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

Core features scope	Four features, described at a UI level ("the app compares reports and shows status")	Five features, each with named Flutter widgets and an hour estimate; route status split out as its own line	After completing M4A4 and M5A5, I had a better understanding of the amount of work involved in stateful screens, forms, lists, navigation, and external packages. I therefore separated authentication and route-status calculation into explicit implementation items so that each part could receive its own realistic estimate rather than hiding significant work inside broader feature descriptions.
Route representation	A saved route would use a real road-following polyline generated through the Google Directions API	Two tapped map points (start, destination), no external routing API	I removed the external routing dependency to keep the MVP achievable. The simplified representation still demonstrates saving a route and checking it against flood reports.
Maps	Google Maps SDK + Directions + Places + Geocoding APIs	flutter_map + OpenStreetMap + latlong2	This avoids a billing-enabled Google Maps/Directions setup while providing the map, markers, and coordinate handling required by the MVP.
Data model	Route/Flood Report/User fields oriented around Serverpod's model classes and an encoded polyline field	Supabase PostgreSQL tables (profiles, routes, flood_reports) with plain start/destination coordinate fields, no polyline field	The data model was updated to match Supabase and the simplified route representation. Routes now store start and destination coordinates instead of an encoded polyline.
Flood-report photos	Optional photos included in the flood-report feature	Photo attachment moved outside the core MVP	I moved photo upload to the stretch goals because flood reporting works without it, reducing implementation risk

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

			while keeping the feature possible later.
Risks	One risk: route/flood matching	Two risks: route/flood matching (now smaller in scope, still unresolved) plus a new Supabase/backend integration risk	The route-matching risk remains, although the simplified route makes it smaller. Supabase adds a new risk involving authentication, database relationships, RLS, and asynchronous queries.
Runnability	Not addressed	Device Preview retained, browser execution planned, hardware fallbacks defined, secrets identified	The revised requirements explicitly ask how another person can run the project, so I added a concrete runnability plan.
Screens	Five screens: Sign In/Guest, Home, Add Route, Route Details, Report Flood	Same five screens, unchanged. Home contains two presentation states (List and Map) and a profile/account menu overlay, but neither creates an additional screen.	Finalizing the mockup revealed that Home needed a clear sign-out/switch-account path. Adding the account menu as an overlay solves that navigation gap while preserving the original five-screen structure.

Final Consistency Check

- MVP features agree with screens — yes. The five MVP features are represented across the five screens, although the relationship is not strictly one-feature-to-one-screen. Authentication is handled by Sign In / Guest Entry; Home supports the saved-route and nearby-report workflow; Add Route handles route creation; Route Details handles route-status checking; and Report Flood handles flood-report submission.
- Screens agree with the mockup/wireframes — yes. The final mockup uses the same five screens and purposes: Sign In / Guest Entry, Home, Add Route, Route Details, and Report

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

Flood. Home's List and Map views are two states of the same screen, while the profile/account menu is an overlay rather than a sixth screen.

- Every saved field appears in the data table — yes. The fields defined in the Supabase schema in Section 6 are represented in Section 9.
- Every data-table item has a Supabase location — yes. User profiles, routes, and flood reports are stored in Supabase PostgreSQL; authentication is handled by Supabase Auth; and any future photo files use Supabase Storage.
- Authentication uses Supabase Auth, not Serverpod — yes. The revised proposal consistently uses Supabase Auth for sign-up, sign-in, sign-out, and authentication sessions.
- Flood reports are shared between users — yes. Flood reports are stored in the shared Supabase database so that reports submitted by one user can be viewed by other users.
- Saved routes remain user-specific — yes. Each route contains a user_id identifying its owner, with Row Level Security planned to restrict access appropriately.
- Supabase is consistently named as the chosen backend — yes. Supabase is the selected backend throughout the revised proposal.
- No section accidentally says Firebase is the chosen backend — correct. Firebase is mentioned only as an alternative that was considered and rejected.
- No section recommends shared_preferences as the primary storage — correct. It appears only as an unsuitable alternative for this application's shared-data requirement.
- Stretch goals are separated from the MVP — yes. Section 5 contains features deliberately excluded from the core MVP, including notifications, severity visualization, trusted reporters, advanced map interaction, and photo-upload enhancements.

HOLY ANGEL UNIVERSITY**School of Computing****6ADET**

- Risks include both the original route-matching issue and a new backend/integration issue — yes. Section 11 revisits the original route/flood matching risk and adds Supabase integration as a second risk.
- The one-hour Supabase spike is not falsely claimed as completed — correct. Section 6 explicitly states that it has not yet been completed and gives August 29, 2026 as the target date.
- Web runnability is explicitly addressed — yes. Section 8 explains the planned flutter run -d web-server verification and identifies what still needs to be tested.
- Hardware-dependent functionality has a fallback/demo plan — yes. GPS can fall back to manual map selection, while real GPS behavior will be demonstrated on an Android device.
- Public GitHub secrets are addressed — yes. Section 8 identifies the Supabase URL/public key, explains that the service-role key must never be exposed, and states that no real personal data will be committed.
- The change log shows meaningful revisions from the preliminary proposal — yes. Section 12 documents the backend, authentication, MVP scope, route representation, mapping implementation, data model, risks, runnability, and screen decisions with specific reasons for the changes.